

Arquitetura e Decisões Técnicas

Stack, arquitetura de componentes, multi-tenant, dados e telemetria, inteligência artificial, infraestrutura, integrações, segurança e custos operacionais da plataforma inteligente de gestão de frotas.

ARQUITETURA

DECISÕES

INFRAESTRUTURA

PRODUTO

RookHub

VERSÃO

1.0

DATA

Julho de 2026

BASE

PRD RookHub 1.0

PÚBLICO

Engenharia, Dados,
Infra e Produto

Sumário

0	Controle do Documento	5
0.1	Como ler este documento	5
0.2	Status das decisões	5
1	Sumário Executivo	6
1.1	Stack em uma tabela	6
2	Visão Geral da Arquitetura	8
2.1	Diagrama de componentes	8
2.2	Fluxos críticos	9
3	Bloco 1 — Frontend, Mobile e Design System	10
3.1	Estrutura do monorepo	10
FE-01	Fonte de verdade da paleta: prosa do DESIGN.md	10
FE-02	Raio de cantos: 16px / 12px	10
FE-03 / FE-04	Separação painel x site institucional	11
FE-05	PWA do motorista como aplicação independente	11
FE-06	Tailwind CSS v4 + shadcn/ui + packages/ui	12
FE-07	Fallback de blur e gate de acessibilidade no CI	13
FE-08	Estado e camada offline	14
FE-09	Gráficos: híbrido visx + Recharts	15
FE-10	Mapas: Mapbox GL JS	15
FE-11	Tempo real: WebSocket	15
FE-12	Paleta completa (rampa OKLCH)	16
4	Bloco 2 — Backend, APIs e Multi-tenant	20
BE-01 / BE-02 / BE-03	Linguagens e time	20
BE-09	Spring Boot 3.x + Spring Modulith	20
BE-04	API REST + OpenAPI 3.1	21

BE-10	Persistência híbrida: Spring Data JPA + jOOQ	22
BE-14	Isolamento multi-tenant: implementação do RLS	22
BE-11	Autenticação própria com Spring Security	24
BE-12 / BE-13	Processamento assíncrono	25
BE-08	Monólito modular + worker dedicado	26
5	Bloco 3 — Dados, Telemetria e Armazenamento	28
DAT-01a	PostgreSQL 16 + TimescaleDB no Timescale Cloud (sobre AWS)	28
DAT-02	Telemetria em hypertables	28
DAT-06	Agregações: continuous aggregates + cache Redis	29
DAT-03	Cloudflare R2 para objetos	30
DAT-04	Copiloto do operador sem pipeline de vídeo	31
DAT-05	Dataset de treinamento (RN-082)	32
DAT-07	Log de auditoria imutável (RNF-020)	32
5.1	Modelo de dados — convenções obrigatórias	33
6	Bloco 4 — Inteligência Artificial	34
IA-01	Google Gemini via API direta (camada paga)	34
IA-02	Orquestração com LangChain4j	34
IA-04	Pipeline do assistente e governança	35
IA-03	Voz: Google STT + ElevenLabs TTS	36
IA-05	Vector DB: pgvector, ativação condicional	37
7	Bloco 4 — Infraestrutura e DevOps	38
INF-01	AWS ECS Fargate + ALB	38
INF-02	Hospedagem dos frontends	38
INF-03	CI/CD, IaC, ambientes e migrations	39
INF-04	Observabilidade e serviços de apoio	40
8	Integrações	43
INT-01	Camada de anticorrupção obrigatória (RN-138)	43

INT-02	Padrão híbrido (RN-137)	43
INT-03	Observabilidade de integração	44
9	Segurança	45
9.1	LGPD (Seção 19 do PRD)	45
10	Custos Operacionais Estimados	47
10.1	MVP — 2 tenants, ~80 veículos	47
10.2	Análise	48
11	Riscos Técnicos e Dívidas Assumidas	49
12	Rastreabilidade — Decisão x Requisito	51
13	Definição de Pronto — Complemento Técnico	53
14	Próximos Passos Recomendados	54
	Antes da sprint 1 (bloqueantes)	54
	Sprint 0 — fundação técnica (2 semanas sugeridas)	54

0 Controle do Documento

Quadro 1 — Controle do Documento

CAMPO	VALOR
Produto	RookHub
Documento	Arquitetura e Decisões Técnicas
Versão	1.0
Status	Aprovado para desenvolvimento do MVP
Data	28 de julho de 2026
Autor	Lead Solutions Architect / CTO
Documentos de origem	prd_RookHub.md v1.0, DESIGN.md , vi-sao_e_escopo_negocio_RookHub.md
Público-alvo	Engenharia (Backend, Frontend, Mobile, Dados, DevOps), QA, Design, Segurança

Fonte: elaboração própria, RookHub (2026).

0.1 Como ler este documento

- FE-xx — Decisão de Frontend
- BE-xx — Decisão de Backend
- DAT-xx — Decisão de Dados e Armazenamento
- IA-xx — Decisão de Inteligência Artificial
- INF-xx — Decisão de Infraestrutura e DevOps
- DT-xx — Dívida Técnica assumida conscientemente
- RT-xx — Risco Técnico

Toda decisão registrada é rastreável ao requisito do PRD que a motivou (Seção 12).

0.2 Status das decisões

Todas as decisões deste documento foram apresentadas com alternativas, prós, contras e impacto financeiro, e **aprovadas explicitamente pelo tomador de decisão** em sessão de arquitetura. Nenhuma escolha foi feita unilateralmente pela consultoria.

1 Sumário Executivo

O RookHub é um SaaS B2B multi-tenant para gestão de frotas, com requisitos simultâneos de:

- **Alta densidade informacional** com identidade visual sofisticada (glassmorphism total)
- **Operação offline integral** no aplicativo do motorista
- **Ingestão contínua de telemetria** (~7–14M pontos/mês por tenant)
- **Isolamento multi-tenant rigoroso** com Row-Level Security
- **Assistente de IA generativa** com garantia absoluta de não-vazamento de dados por RBAC
- **Baixo custo operacional** compatível com estágio pré-escala

A arquitetura escolhida é um **monólito modular em Java/Spring Boot com worker dedicado**, sobre **PostgreSQL + TimescaleDB**, servindo **três aplicações frontend independentes em monorepo**, hospedado em **AWS ECS Fargate**, com IA orquestrada por **LangChain4j sobre Google Gemini**.

O princípio arquitetural condutor é: **maximizar velocidade de entrega de regra de negócio (148 RNs no PRD) e minimizar superfície operacional (time de 3 desenvolvedores backend)**, sem comprometer o isolamento de dados, que é o requisito não-negociável do produto.

1.1 Stack em uma tabela

Quadro 2 — Stack em uma tabela

CAMADA	TECNOLOGIA
Painel web	Vite + React 19 + React Router v7 (SPA)
PWA do motorista	Vite + React 19 + Workbox + Dexie (IndexedDB)
Site institucional	Next.js 15 (SSR/SSG)
Estilização	Tailwind CSS v4 + shadcn/ui + tokens do DESIGN.md
Gráficos	visx (Painel do Dono) + Recharts (demais)
Mapas	Mapbox GL JS
Tempo real	WebSocket (STOMP sobre SockJS)

CAMADA	TECNOLOGIA
Backend	Java 21 LTS + Spring Boot 3.x + Spring Modulith
API	REST + OpenAPI 3.1 (cliente TS gerado)
Persistência	Spring Data JPA (CRUD) + jOOQ (analítico)
Banco principal	PostgreSQL 16 + TimescaleDB (Timescale Cloud sobre AWS)
Cache / filas / pub-sub	Redis (AWS ElastiCache) + Redisson
Processamento em lote	Spring Batch
Object storage	Cloudflare R2
Autenticação	Spring Security (JWT + refresh rotativo, Argon2id)
IA	LangChain4j + Google Gemini (API paga)
Voz	Google Speech-to-Text (STT) + ElevenLabs (TTS)
Computação	AWS ECS Fargate + ALB
IaC	Terraform / OpenTofu
CI/CD	GitHub Actions
Observabilidade	Datadog + Sentry

Fonte: elaboração própria, RookHub (2026).

2 Visão Geral da Arquitetura

2.1 Diagrama de componentes

Diagrama 1 — Diagrama de componentes



2.2 Fluxos críticos

Fluxo 1 — Checklist offline do motorista (RF-017)

Trecho de referência 2 — Fluxos críticos

```
Motorista preenche offline → Dexie (IndexedDB) grava filled_at (relógio do device)
  → fotos comprimidas a ~300KB / 1600px (RN-040) via canvas
  → fila de sincronização persistente (7 dias / 20 checklists / 100 fotos)
  → ao voltar online: dados estruturados primeiro (RNF-011)
  → POST /v1/checklists (idempotente por client_uuid)
  → backend grava received_at; divergência > 6h ⇒ flag de auditoria (RN-054)
  → fotos sobem em background via Background Sync API → presigned PUT no R2
  → conflito: servidor é autoridade (RN-053); app exibe aviso
```

Fluxo 2 — Evento crítico de segurança até o toast (RNF-007, < 10s)

Trecho de referência 3 — Fluxos críticos

```
Fornecedor → webhook → ACL do fornecedor traduz p/ modelo canônico (RN-138)
  → fila Redis (idempotência por event_id externo, RN-142)
  → worker: classifica severidade (tabela fixa RookHub, RN-083)
  → severidade Crítico? → publica em Redis pub/sub
  → instância(s) api entregam via WebSocket ao MANAGER/OWNER do tenant
  → toast com ações embutidas: "Ligar para o motorista" | "Abrir viagem" (RN-091)
  → demais severidades: agregação em janela configurável (RN-089, default 15min)
```

Fluxo 3 — Pergunte à Sua Frota (RF-033 a RF-037)

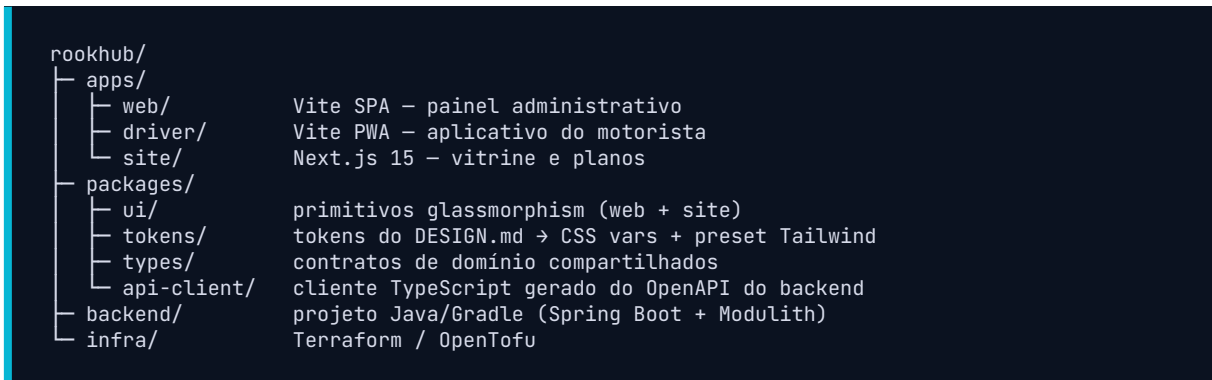
Trecho de referência 4 — Fluxos críticos

```
Ctrl+K ou microfone
  → [STT Google, se voz]
  → POST /v1/assistant/ask
  → LangChain4j: classificação de intenção + function calling na mesma chamada
  → GATE DE AUTORIZAÇÃO no backend, ANTES da execução (RN-119):
    entitlement do tenant? · permissão do papel? · operator_sees_financials?
    x ⇒ função não é chamada; recusa educada
  → execução da função determinística (j00Q, dentro de withTenant)
  → tokenização do payload (RN-122): nomes/placas/CPF → tokens opacos
  → Gemini recebe apenas números + tokens
  → re-hidratação no backend + fonte e período (RN-121)
  → resposta: texto + gráfico/tabela + ação contextual (RN-116)
  → [TTS ElevenLabs em streaming, se voz]
```

3 Bloco 1 — Frontend, Mobile e Design System

3.1 Estrutura do monorepo

Diagrama 5 — Estrutura do monorepo



Ferramenta: Turborepo + pnpm workspaces.

FE-01 Fonte de verdade da paleta: prosa do DESIGN.md

Contexto. O `DESIGN.md` continha duas paletas conflitantes: o front-matter YAML (aparentemente gerado por ferramenta Material 3, com `primary #c0c1ff` e `background #13131b`) e a prosa da seção "Colors" (`Indigo #6366F1`, `Cyan #06B6D4`, `Midnight #0B1220`).

Decisão. A **prosa** é a fonte de verdade, por representar a intenção de marca. O YAML é descartado.

Consequência. A prosa definia apenas 3 cores e um gradiente, contra ~50 tokens do YAML. Foi necessário gerar a rampa tonal completa ([FE-12](#)).

FE-02 Raio de cantos: 16px / 12px

- **Containers principais** (cards, painéis, modais): `16px` — `rounded-lg`
- **Elementos internos** (botões, inputs, chips, itens aninhados): `12px` — `rounded-md`

Resolve a contradição entre `rounded.lg = 1rem` no YAML e "20px" na prosa.

FE-03 / FE-04 Separação painel × site institucional

Quadro 3 — Separação painel × site institucional

APP	TECNOLOGIA	JUSTIFICATIVA
apps/web	Vite + React 19 SPA	Aplicação 100% autenticada, sem necessidade de SEO. SSR agregaria complexidade sem benefício. Deploy estático em CDN, custo próximo de zero, sem servidor Node em produção.
apps/site	Next.js 15	Vitrine do produto e página de planos exigem SEO, SSG e boa performance de primeira carga. Permite ao time de marketing operar com autonomia.

Fonte: elaboração própria, RookHub (2026).

Alternativas descartadas: Next.js unificado (SSR desnecessário no painel), Nuxt/Vue (ecossistema de bibliotecas de dados menor), Remix/TanStack Start (menor massa crítica de contratação no Brasil).

Mitigação do RNF-001 (< 2s p95) sem SSR: agregações pré-calculadas no backend ([DAT-06](#)), cache client-side agressivo com TanStack Query, code-splitting por rota, prefetch em hover e skeletons que refletem o layout final.

FE-05 PWA do motorista como aplicação independente

Decisão. [apps/driver](#) é uma aplicação separada dentro do monorepo, com bundle, service worker e deploy próprios.

Justificativa. Os requisitos do motorista são **antagônicos** aos do painel:

Quadro 4 — PWA do motorista como aplicação independente

REQUISITO	PAINEL	PWA DO MOTORISTA
Estética	Glassmorphism total	RNF-029: abandona o glassmorphism — legibilidade sob sol
Rede	Banda larga	3G em rodovia
Offline	Não requerido	Integral (RN-052)
Bundle	Gráficos, mapas, IA	Deve ser mínimo
Alvos de toque	Densidade alta	Grandes (operação com luvas)

Fonte: elaboração própria, RookHub (2026).

Um bundle único faria o motorista baixar código de gráficos, mapas e IA que jamais usará, em conexão 3G — inaceitável.

Câmera nativa (RN-041). Implementada via `<input type="file" accept="image/*" capture="environment">`, que abre a câmera diretamente e não expõe a galeria.

RT-01 Limitações de PWA em iOS

Quadro 5 — Limitações de PWA em iOS

RISCO	IMPACTO	MITIGAÇÃO
Web Push exige iOS 16.4+ e instalação na tela de início	Motorista com iPhone antigo não recebe push	Levantar a distribuição Android/iOS da frota-âncora antes da sprint 1. Fallback: SMS via Zenvia para eventos críticos.
Cota de IndexedDB mais restritiva; Safari pode expurgar dados após 7 dias sem uso	Perda de fila offline	RNF-010 exige 7 dias — exatamente o limite. Monitorar <code>navigator.storage.estimate()</code> e solicitar armazenamento persistente.
<code>capture</code> tem comportamento inconsistente em versões antigas do Safari	Galeria pode ser exposta	Validação server-side de metadados EXIF de captura recente

Fonte: elaboração própria, RookHub (2026).

Ação obrigatória antes da sprint 1: confirmar a distribuição de sistemas operacionais na frota-âncora.

FE-06 Tailwind CSS v4 + shadcn/ui + packages/ui

Decisão. Tailwind v4 (motor Oxide, configuração via `@theme` em CSS) com componentes shadcn/ui (Radix + Tailwind, código copiado para o repositório).

Justificativa. O `DESIGN.md` exige que **todo** container tenha `background: rgba(255,255,255,0.04)`, `backdrop-filter: blur(16px)` e borda de 1px em gradiente. Isso não pode ser uma classe repetida centenas de vezes — precisa ser um componente. O shadcn/ui entrega acessibilidade Radix com código sob nosso controle, essencial para reescrever visualmente todos os primitivos.

Alternativas descartadas: Mantine/MUI (sobrescrever tema completo para glassmorphism é mais caro que construir), CSS Modules (sem design system pronto), Panda/vanilla-extract (ecossistema menor).

Primitivos de `packages/ui`:

Trecho de referência 6 — Tailwind CSS v4 + shadcn/ui + packages/ui

```
GlassCard · GlassPanel · GlassModal · GlassInput · GlassSelect
SpectrumButton · GhostButton · Chip · Badge
DataTable (Inter tabular — RNF-027)
FleetIndicator (glow pulsante — Ativo/0cioso)
GlowBackdrop (radiais orgânicas de canto)
```

Implementação do vidro**Trecho CSS 7 — Implementação do vidro**

```
.glass {
  background: rgba(255, 255, 255, 0.04);
  backdrop-filter: blur(16px);
  border-radius: 16px;
  border: 1px solid transparent;
  background-image:
    linear-gradient(rgba(255,255,255,0.04), rgba(255,255,255,0.04)),
    linear-gradient(135deg, rgba(255,255,255,0.15), rgba(255,255,255,0.05));
  background-origin: border-box;
  background-clip: padding-box, border-box;
}

.glass-elevated { /* modais — DESIGN.md */
  background: rgba(255, 255, 255, 0.08);
  backdrop-filter: blur(24px);
}
```

FE-07 Fallback de blur e gate de acessibilidade no CI

Problema. `backdrop-filter: blur()` é uma das operações mais caras do compositor. Com dezenas de painéis simultâneos, derruba o FPS em Android intermediário e em notebooks corporativos com GPU integrada — exatamente o hardware do operador de escritório, cuja produtividade é o **princípio inegociável** do produto (RNF-006).

Decisão.

- 1. Fallback automático de blur** - `@supports not (backdrop-filter: blur(1px))` ⇒ superfície sólida opaca - Detecção de dispositivo de baixa performance (`navigator.hardwareConcurrency`, `deviceMemory`) ⇒ desativa blur globalmente via classe raiz - **Regra de projeto:** listas e tabelas longas (>20 linhas) **nunca** usam blur por item — apenas o container externo - Preferência do usuário persistida ("modo alto desempenho") - `prefers-reduced-motion` desativa os glows pulsantes de `FleetIndicator`
- 2. Gate de contraste no CI (RNF-028)** - `axe-core` via Playwright em todas as rotas principais - Build **falha** em qualquer violação de contraste WCAG AA - Como o contraste sobre vidro depende do que está atrás, a opacidade mínima da camada de vidro é

travada em `0.04` sobre superfícies escuras conhecidas, tornando o contraste calculável e determinístico

FE-08 Estado e camada offline

Quadro 6 — Estado e camada offline

CAMADA	ESCOLHA
Server state	TanStack Query v5 — cache, retry, optimistic updates, invalidação
Client state	Zustand — leve, sem boilerplate
Offline (motorista)	Dexie.js (IndexedDB) + fila de sincronização própria
Service worker	Workbox — precache do shell, Background Sync para fotos

Fonte: elaboração própria, RookHub (2026).

Justificativa da fila própria. As regras do PRD são específicas demais para uma engine genérica de sincronização: dois timestamps distintos (RN-054), prioridade de dados estruturados sobre fotos (RNF-011), servidor como autoridade em conflito (RN-053), capacidade dimensionada (RNF-010) e indicador visual permanente de estado (RNF-013). RxDB foi descartado por adicionar custo de licença sem resolver esses pontos; PouchDB/CouchDB foi descartado por exigir um banco incompatível com a estratégia PostgreSQL + RLS.

Esquema do Dexie

Trecho JavaScript 8 — Esquema do Dexie

```
db.version(1).stores({
  checklists: '++id, uuid, status, filled_at, vehicle_id',
  photos: '++id, checklist_item_uuid, blob, uploaded',
  syncQueue: '++id, type, priority, attempts, created_at',
  vehicleData: 'id, updated_at', // pendências abertas p/ consulta offline
  session: 'key' // um motorista por aparelho — RNF-012
});
```

Idempotência. Todo registro criado offline carrega um `client_uuid` (UUID v7) gerado no dispositivo. O backend usa esse campo como chave de idempotência, tornando o reenvio seguro.

FE-09 Gráficos: híbrido visx + Recharts

Quadro 7 — Gráficos: híbrido visx + Recharts

CONTEXTO	BIBLIO-TECA	JUSTIFICATIVA
Painel do Dono (4 blocos de vitrine, RF-032)	visx / D3	Assinatura visual, integração com glassmorphism, gradientes de marca. É a tela que o dono vê e que a área comercial demonstra.
Respostas do assistente (RN-116), telas internas, relatórios	Re-charts	Gráficos gerados dinamicamente a partir de dados variáveis exigem componentes polimórficos e prontos.

Fonte: elaboração própria, RookHub (2026).

Regra de fronteira: visx apenas em widget fixo e desenhado; qualquer gráfico gerado por dados dinâmicos usa Recharts. Evita que a dupla dependência vire inconsistência.

FE-10 Mapas: Mapbox GL JS

Justificativa. Estilo dark customizável coerente com o [DESIGN.md](#), boa performance com marcadores animados (indicadores pulsantes de frota), SDK maduro.

RT-02 — Risco de custo. O modelo é por map load. Com 80 veículos em atualização contínua e vários operadores com o mapa aberto durante o expediente, o custo pode escalar de forma não-linear.

Mitigações obrigatórias: - Uma única instância de mapa por sessão (nunca remontar o componente) - Atualização de posição via `setData` na fonte GeoJSON, **jamais** recarregando o mapa - Alerta de orçamento na conta Mapbox - **Gatilho de reavaliação:** se o custo mensal ultrapassar US\$100, migrar para MapLibre GL + tiles Protomaps/MapTiler (API quase idêntica, migração de baixo custo)

FE-11 Tempo real: WebSocket

Decisão. WebSocket com STOMP sobre SockJS (Spring WebSocket), com fallback automático.

Consequências arquiteturais assumidas:

- 1. Elimina serverless da API** — Vercel Functions, Lambda e Cloudflare Workers ficam fora para o backend, pois exigiriam conexão persistente. Isso condicionou [INF-01](#) (ECS Fargate).

- 2. Exige Redis pub/sub — com múltiplas tasks, uma notificação gerada no worker precisa alcançar o cliente conectado a qualquer instância da API. Resolvido por `ReLayBroker` sobre Redis (Redisson).
- 3. Exige sticky sessions no ALB.
- 4. Exige heartbeat e reconexão com ressincronização de estado ao religar.

Tópicos:

Trecho de referência 9 — Tempo real: WebSocket

```
/topic/tenant.{tenantId}.notifications      → Central de Notificações (RF-038)
/topic/tenant.{tenantId}.safety.critical    → toasts críticos (RN-087)
/topic/tenant.{tenantId}.fleet.status       → estado de veículos
/user/queue/personal                       → notificações individuais
```

Autorização no handshake: o JWT é validado antes da inscrição, e a inscrição em tópico de tenant diferente do próprio é **rejeitada** — o RBAC vale também no canal de tempo real.

FE-12 Paleta completa (rampa OKLCH)

Gerada a partir das âncoras imutáveis `#6366F1` (Indigo), `#06B6D4` (Cyan) e `#0B1220` (Midnight), com matiz azulado (H ≈ 250°) preservado em toda a rampa neutra para coerência sob camadas de vidro.

Superfícies

Quadro 8 — Superfícies

TOKEN	HEX	USO
<code>background</code>	<code>#0B1220</code>	Void atmosférico (âncora)
<code>surface</code>	<code>#0B1220</code>	Base
<code>surface-container-lowest</code>	<code>#070C16</code>	Poços, fundos de input
<code>surface-container-low</code>	<code>#111827</code>	Vidro sobre void
<code>surface-container</code>	<code>#161F31</code>	Card padrão (base do glass 0.04)
<code>surface-container-high</code>	<code>#1C273B</code>	Card elevado
<code>surface-container-highest</code>	<code>#243146</code>	Modais (base do glass 0.08)

Fonte: elaboração própria, RookHub (2026).

Texto e traço

Quadro 9 — Texto e traço

TOKEN	HEX	CONTRASTE SOBRE #0B1220
on-surface	#E8EAF2	15.8:1 — AAA
on-surface-variant	#A8B0C4	8.1:1 — AAA
on-surface-muted	#7B849B	4.9:1 — AA
outline	#4A5468	3:1 (traço de componente)
outline-variant	#232C3E	divisórias sutis

Fonte: elaboração própria, RookHub (2026).

Marca e semânticos

Quadro 10 — Marca e semânticos

TOKEN	HEX	OBSERVAÇÃO
primary	#6366F1	âncora — Indigo
primary-container	#3730A3	fundo de estado ativo
on-primary	#FFFFFF	4.6:1 — AA
secondary	#06B6D4	âncora — Cyan
secondary-container	#0E7490	
on-secondary	#00212B	
tertiary	#0B1220	Midnight estrutural (âncora)
success	#34D399	veículo disponível, plano em dia
warning	#FBBF24	severidade Atenção (RN-032)
error	#FB7185	severidade Crítico , veículo bloqueado
info	#38BDF8	

Fonte: elaboração própria, RookHub (2026).

DESTAQUE

Nota de acessibilidade. Os semânticos foram deliberadamente escolhidos na faixa **clara** de cada matiz. Um vermelho escuro convencional (`#DC2626`) reprovava em WCAG AA sobre superfície translúcida escura. Esta escolha atende diretamente ao RNF-028.

Spectrum Gradient (variação magenta — aprovada)**Trecho CSS 10 — Spectrum Gradient (variação magenta — aprovada)**

```
--spectrum-gradient: linear-gradient(135deg,  
  #E879F9 0%,      /* 1 — Magenta      */  
  #A855F7 17%,     /* 2 — Púrpura      */  
  #6366F1 33%,     /* 3 — Indigo       ← âncora */  
  #4F7DF3 50%,     /* 4 — Azul         */  
  #3B9EF5 67%,     /* 5 — Sky          */  
  #06B6D4 83%,     /* 6 — Cyan        ← âncora */  
  #22D3EE 100%    /* 7 — Cyan claro  */  
);
```

Aplicações previstas: fundo da sidebar, hover do botão primário, light leak nas bordas do viewport. Uso **parcimonioso**, conforme o `DESIGN.md`.

Glows orgânicos**Trecho CSS 11 — Glows orgânicos**

```
--glow-indigo: radial-gradient(circle, rgba(99,102,241,0.15) 0%, transparent 70%);  
--glow-cyan:   radial-gradient(circle, rgba(6,182,212,0.15) 0%, transparent 70%);
```

Posicionados nos cantos do viewport, com `filter: blur(100px)`, atrás das camadas de vidro. Desativados em dispositivos de baixa performance (`FE-07`).

Tipografia**Quadro 11 — Tipografia**

ESTILO	FONTE	TAMANHO / PESO / ENTRELINHA
<code>display-lg</code>	Sora	48 / 700 / 56 / -0.02em
<code>headline-lg</code>	Sora	32 / 600 / 40
<code>headline-lg-mobile</code>	Sora	28 / 600 / 36

ESTILO	FONTE	TAMANHO / PESO / ENTRELINHA
headline-md	Sora	24 / 600 / 32
body-lg	Inter	18 / 400 / 28
body-md	Inter	16 / 400 / 24
label-md	Inter	14 / 500 / 20 / 0.05em

Fonte: elaboração própria, RookHub (2026).

Números em tabelas (RNF-027): `font-variant-numeric: tabular-nums lining-nums`.

Ícones: Phosphor Icons Duotone.

Carregamento: fontes auto-hospedadas (WOFF2, subset latin + latin-ext), `font-display: swap`, preload dos pesos críticos. Nunca via CDN de terceiros — reduz latência e exposição sob LGPD.

4 Bloco 2 — Backend, APIs e Multi-tenant

BE-01 / BE-02 / BE-03 Linguagens e time

Quadro 12 — Linguagens e time

ITEM	DECISÃO
Linguagem principal	Java — toda regra de negócio, APIs, integrações e IA no MVP
Linguagem secundária	Python — reservado para automações e para o modelo próprio de visão computacional [Fase 2] . Fora do escopo do MVP.
Versão	Java 21 LTS
Time backend	3 pessoas

Fonte: elaboração própria, RookHub (2026).

Nota sobre virtual threads. Java 21 traz `virtual threads` (Project Loom), que tornam o modelo bloqueante tradicional viável para milhares de conexões concorrentes. **Programação reativa (WebFlux) está descartada:** com 3 desenvolvedores e 148 regras de negócio, o custo de depuração de stack traces reativos não se paga. Habilitar `spring.threads.virtual.enabled=true`.

BE-09 Spring Boot 3.x + Spring Modulith

Justificativa.

- Padrão absoluto de mercado Java, com contratação viável no Brasil
- Spring Security cobre nativamente a dupla camada de autorização da RF-002
- **Spring Modulith** impõe as fronteiras entre módulos **em tempo de build**: se o módulo `costs` importar uma classe interna de `safety`, o teste de arquitetura falha. Com 9 módulos e 3 desenvolvedores, isso transforma "monólito modular" de intenção em garantia verificável.
- Eventos de aplicação do Modulith permitem comunicação entre módulos sem acoplamento direto, facilitando extração futura para serviço próprio se necessário

Alternativas descartadas: Quarkus (ecossistema menor, curva de aprendizado para o time), Micronaut (comunidade reduzida no Brasil).

Módulos

Quadro 13 — Módulos

MÓDULO	RESPONSABILIDADE	REQUISITOS DO PRD
<code>identity</code>	Autenticação, usuários, papéis, sessões, MFA, PIN/QR	RF-003 a RF-007
<code>tenancy</code>	Tenants, planos, entitlements, <code>tenant_modules</code>	RF-001, RF-002
<code>fleet</code>	Veículos, implementos, motoristas, oficinas, fornecedores	RF-008 a RF-010
<code>trips</code>	Viagens e máquina de estados	RF-011
<code>checklist</code>	Templates versionados, preenchimento, pendências, bloqueio	RF-012 a RF-017
<code>costs</code>	Custo por km em camadas, abastecimento, anomalias	RF-018 a RF-022
<code>maintenance</code>	Planos, catálogo, ordens de serviço	RF-023, RF-024
<code>safety</code>	Eventos, contestação, score, copiloto do operador	RF-025 a RF-031
<code>assistant</code>	Intenções, function calling, governança da IA	RF-033 a RF-037
<code>integrations</code>	ACLs por fornecedor, webhooks, polling, idempotência	RF-041, RF-042
<code>notifications</code>	Central global multi-módulo, canais	RF-038
<code>onboarding</code>	Wizard, importação em lote	RF-039, RF-040
<code>audit</code>	Log imutável, hash encadeado	RNF-020

Fonte: elaboração própria, RookHub (2026).

BE-04 API REST + OpenAPI 3.1

Decisão. REST com contrato OpenAPI gerado a partir do código (springdoc-openapi), servindo de fonte para a geração automática do cliente TypeScript em `packages/api-client`.

Justificativa.

- 1. Compatibilidade com a stack.** Java no backend e TypeScript no frontend eliminam tRPC (exige TS nas duas pontas).
- 2. RN-118/RN-119 favorecem RPC/REST sobre GraphQL.** O requisito exige que o assistente não revele dado bloqueado nem de forma agregada ou indireta, com verificação **no backend antes da execução da função**. Autorização por campo em um grafo é signifi-

cativamente mais difícil de auditar do que autorização por endpoint/função nomeada. GraphQL foi descartado por razão de segurança, não de preferência.

3. Integrações e futuro app nativo [Fase 2] consomem REST naturalmente.

Convenções:

- Versionamento por caminho: `/v1/ ...`
- Paginação por cursor em listagens de alto volume
- `Idempotency-Key` obrigatório em POST de criação de lançamento
- Erros no padrão RFC 9457 (Problem Details)
- Rate limiting por tenant e por usuário (RNF-021) via Redisson, com `429` e `Retry-After`

BE-10 Persistência híbrida: Spring Data JPA + jOOQ

Quadro 14 — Persistência híbrida: Spring Data JPA + jOOQ

FERRAMENTA	USO
Spring Data JPA / Hibernate	CRUD de cadastros base, entidades de domínio, relacionamentos simples
jOOQ	Obrigatório em toda consulta analítica: Painel do Dono, custo por km, ranking, score de segurança, e todas as funções consumidas pelo assistente de IA

Fonte: elaboração própria, RookHub (2026).

Justificativa. JPA entrega produtividade em CRUD, mas abstrai o SQL de forma perigosa em consultas analíticas e pode mascarar o contexto de tenant via cache de primeiro nível. jOOQ é SQL-first e type-safe, gerado a partir do schema real, com controle total da transação — indispensável para as agregações que sustentam o RNF-001.

Licença: jOOQ é gratuito para PostgreSQL (Open Source Edition).

BE-14 Isolamento multi-tenant: implementação do RLS

Este é o requisito mais crítico de toda a arquitetura. Uma falha aqui é um incidente de segurança que encerra o produto.

O risco

O PostgreSQL RLS lê o tenant de uma variável de sessão. Com connection pooling, se a variável for definida com `SET` (em vez de `SET LOCAL`) ou fora de uma transação, a conexão retorna ao pool carregando o tenant anterior, e a próxima requisição — de outro cliente — lê dados errados. Esse defeito não aparece em teste unitário; manifesta-se apenas sob concorrência, em produção.

A implementação

1. Policy no banco

Consulta SQL 12 — A implementação

```
ALTER TABLE vehicles ENABLE ROW LEVEL SECURITY;
ALTER TABLE vehicles FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON vehicles
  USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

`FORCE ROW LEVEL SECURITY` garante que a policy se aplique inclusive ao dono da tabela.

2. Ponto único de acesso ao banco

Trecho Java 13 — A implementação

```
@Component
@RequiredArgsConstructor
public class TenantContext {

    private final JdbcTemplate jdbc;

    @Transactional
    public <T> T withTenant(UUID tenantId, Supplier<T> work) {
        Objects.requireNonNull(tenantId, "tenant_id obrigatório (RN-001)");
        jdbc.update("SELECT set_config('app.current_tenant_id', ?, true)",
                    tenantId.toString()); // true = LOCAL, escopo da transação
        return work.get();
    }
}
```

O uso de `set_config(..., true)` (equivalente a `SET LOCAL`) garante que a variável seja descartada ao fim da transação, tornando o retorno ao pool seguro.

3. Quatro camadas de garantia

Quadro 15 — A implementação

CAMADA	MECANISMO
Runtime	<code>TransactionSynchronization</code> que aborta qualquer transação iniciada sem tenant definido, exceto rotas explicitamente marcadas como cross-tenant
Compilação	Teste <code>ArchUnit</code> que falha o build se <code>DataSource</code> , <code>EntityManager</code> ou <code>DSLContext</code> forem injetados fora de <code>TenantContext</code>

CAMADA	MECANISMO
Schema (RN-002)	Job de CI que varre o <code>information_schema</code> e falha o build se qualquer tabela de domínio existir sem <code>tenant_id NOT NULL</code> ou sem policy RLS correspondente. Bloqueante e não desativável por decisão de squad.
Concorrência	Teste de integração que executa operações de dois tenants em paralelo sobre o mesmo pool e assere ausência de vazamento

Fonte: elaboração própria, RookHub (2026).

4. Acesso cross-tenant (RN-003)

Exclusivo do papel `SUPER_ADMIN`, através de método anotado explicitamente, exigindo **motivo textual obrigatório**, e sempre gravando no log de auditoria imutável (usuário, tenant acessado, timestamp, motivo).

5. Entitlements (RF-002)

Duas camadas independentes, ambas verificadas no backend:

Trecho Java 14 — A implementação

```
@RequiresEntitlement(Module.SAFETY)    // o plano contratado inclui?
@PreAuthorize("hasRole('MANAGER')")    // o papel pode executar?
public SafetyReport getSafetyReport(...) { ... }
```

Módulos não contratados aparecem na navegação em estado bloqueado com CTA "Conhecer" (RN-004). Dados de amostra são permitidos **exclusivamente** na tela de Planos.

BE-11 Autenticação própria com Spring Security

Justificativa da escolha por solução própria. Provedores gerenciados (Auth0, Clerk, WorkOS) cobram por usuário ativo mensal. Em um SaaS onde cada tenant traz dezenas de motoristas, isso corrói a margem. Além disso, o fluxo de acesso do motorista (QR code + PIN) é não-padrão e exigiria implementação customizada em qualquer provedor.

Quadro 16 — Autenticação própria com Spring Security

REQUISITO	IMPLEMENTAÇÃO
Senha e PIN (RNF-019)	<code>Argon2PasswordEncoder</code> (Argon2id)
E-mail/senha	<code>DaoAuthenticationProvider</code>

REQUISITO	IMPLEMENTAÇÃO
SSO Google	Spring Security OAuth2 Client
MFA/TOTP (RF-004)	Segredo TOTP por usuário + códigos de recuperação
Sessão (RF-005)	JWT de acesso (15 min) + refresh token rotativo persistido, revogável
Motorista (QR + PIN)	Token de dispositivo de longa duração emitido no wizard (RF-039, etapa 4) + PIN local. Um motorista por aparelho (RNF-012) — o vínculo device↔motorista é único e a troca invalida o anterior.
Rate limiting (RNF-021)	Filtro com Redisson, por tenant e por usuário

Fonte: elaboração própria, RookHub (2026).

Rotação de refresh token com detecção de reuso: se um refresh token já utilizado for reapresentado, toda a família de tokens daquele usuário é revogada (indicador clássico de roubo de token).

BE-12 / BE-13 Processamento assíncrono

Quadro 17 — Processamento assíncrono

COMPONENTE	USO
Redisson	Filas de integração com retry exponencial (RN-137), locks distribuídos, pub/sub para WebSocket, rate limiting
Spring Batch	Importação em lote de planilhas (RN-130): processamento em chunks, skip policy, relatório de erro linha a linha, confirmação antes da gravação
@Scheduled com ShedLock	Tarefas periódicas (polling, digest diário, expurgo de retenção) executadas uma única vez mesmo com múltiplas instâncias

Fonte: elaboração própria, RookHub (2026).

BE-13 — Evolução planejada. Quando o volume de telemetria justificar (indicador: > 50M pontos/mês ou latência de fila > 30s no p95), migrar a ingestão para **Kafka (AWS MSK)**. A camada de filas deve ser abstraída por interface desde o MVP para que essa migração não toque a regra de negócio.

Filas do MVP

Quadro 18 — Filas do MVP

FILA	PRIORIDADE	CONTEÚDO
<code>integration.webhook</code>	Alta	Eventos recebidos de fornecedores
<code>integration.poll</code>	Média	Resultados de polling agendado
<code>safety.event</code>	Crítica	Eventos de segurança (RNF-007, < 10s)
<code>notification.dispatch</code>	Alta	E-mail, push, in-app
<code>cost.recalculate</code>	Baixa	Recálculo de custo por km
<code>media.process</code>	Baixa	Pós-processamento de fotos, geração de Parquet

Fonte: elaboração própria, RookHub (2026).

Idempotência (RN-142): chave composta (`tenant_id`, `provider`, `external_event_id`) com constraint única. Reprocessamento é seguro por construção.

Retry exponencial: 1s → 2s → 4s → 8s → 16s → 32s, máximo de 6 tentativas, seguido de dead letter queue com alerta no Datadog.

BE-08 Monólito modular + worker dedicado

Decisão. Uma única base de código e uma única imagem Docker, com dois serviços ECS distintos por profile Spring:

Quadro 19 — Monólito modular + worker dedicado

SERVIÇO	PROFILE	RESPONSABILIDADE
<code>api</code>	<code>api</code>	Requisições HTTP e WebSocket
<code>worker</code>	<code>worker</code>	Filas, jobs agendados, Spring Batch

Fonte: elaboração própria, RookHub (2026).

Justificativa. Isola o processamento pesado do processo que atende requisições, protegendo o **RNF-004** (< 500ms no salvamento do operador). Uma importação de 5.000 linhas não pode degradar a experiência de quem está lançando dados — o princípio inegociável do produto.

Microserviços foram descartados: com 148 regras de negócio fortemente inter-relacionadas, transações distribuídas trariam complexidade e latência incompatíveis com um MVP de 3 desenvolvedores.

5 Bloco 3 — Dados, Telemetria e Armazenamento

DAT-01a PostgreSQL 16 + TimescaleDB no Timescale Cloud (sobre AWS)

Contexto do conflito. A escolha inicial pelo AWS RDS era incompatível com a escolha pelo TimescaleDB: a AWS não oferece a extensão Timescale em RDS ou Aurora, e não a oferecerá (produtos concorrentes).

Decisão. Timescale Cloud, provisionado na mesma região AWS da aplicação ([sa-east-1](#)), com VPC peering.

Quadro 20 — PostgreSQL 16 + TimescaleDB no Timescale Cloud (sobre AWS)

CRITÉRIO	RESULTADO
Extensão Timescale	Completa, gerenciada pelos criadores
Backup e PITR	Inclusos
Latência para o ECS	Mínima (mesma região)
RPO 24h / RTO 4h (RNF-023/024)	Atendidos com folga (PITR contínuo)
Fatura	Separada da AWS — assumido

Fonte: elaboração própria, RookHub (2026).

Alternativas descartadas: RDS puro com particionamento nativo (perderia continuous aggregates e compressão, gerando dívida técnica significativa), Aiven (mais caro sem ganho), Timestream/InfluxDB (banco separado, sem RLS nem JOIN com o relacional — incompatível com [RBAC-01](#)).

Uma única instância compartilhada, com isolamento por RLS, conforme decisão [RBAC-01](#) do PRD.

DAT-02 Telemetria em hypertables

Volume estimado: 80 veículos × posição a cada 30–60s ≈ 7 a 14 milhões de pontos/mês por tenant.

Consulta SQL 15 — Telemetria em hypertables

```
CREATE TABLE telemetry_points (  
  time          TIMESTAMPTZ      NOT NULL,  
  tenant_id     UUID              NOT NULL,
```

```
vehicle_id    UUID                NOT NULL,
latitude      DOUBLE PRECISION,
longitude     DOUBLE PRECISION,
speed_kmh     REAL,
odometer_km   DOUBLE PRECISION,
fuel_level    REAL,
engine_hours  DOUBLE PRECISION,
provider      TEXT                NOT NULL,
raw_payload   JSONB
);

SELECT create_hypertable('telemetry_points', 'time',
                        chunk_time_interval => INTERVAL '7 days');

ALTER TABLE telemetry_points ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON telemetry_points
  USING (tenant_id = current_setting('app.current_tenant_id')::uuid);

-- Compressão automática após 30 dias (~90% de redução)
ALTER TABLE telemetry_points SET (
  timescaledb.compress,
  timescaledb.compress_segmentby = 'tenant_id, vehicle_id'
);
SELECT add_compression_policy('telemetry_points', INTERVAL '30 days');

-- Retenção de 5 anos (RN-145 – dados operacionais)
SELECT add_retention_policy('telemetry_points', INTERVAL '5 years');
```

RLS funciona normalmente em hypertables — verificado como premissa desta decisão.

DAT-06 Agregações: continuous aggregates + cache Redis

Princípio. Custo por km, consumo e rankings **nunca** são calculados em tempo de requisição. O RNF-001 (< 2s no p95) não sobrevive a isso.

Consulta SQL 16 — Agregações: continuous aggregates + cache Redis

```
CREATE MATERIALIZED VIEW vehicle_daily_metrics
WITH (timescaledb.continuous) AS
SELECT
  time_bucket('1 day', time) AS day,
  tenant_id,
  vehicle_id,
  max(odometer_km) - min(odometer_km) AS km_traveled,
  avg(speed_kmh) AS avg_speed,
  count(*) AS point_count
FROM telemetry_points
GROUP BY day, tenant_id, vehicle_id;

SELECT add_continuous_aggregate_policy('vehicle_daily_metrics',
  start_offset => INTERVAL '7 days',
```

```
end_offset      ⇒ INTERVAL '1 hour',
schedule_interval ⇒ INTERVAL '1 hour');
```

Camadas:

1. **Continuous aggregates** — métricas diárias por veículo, atualizadas incrementalmente
2. **Tabelas de sumário** — custo por km em camadas A/B/C (RN-055), recalculadas por evento na fila `cost.recalculate`
3. **Cache Redis** — resposta do Painel do Dono com TTL de 5 minutos, invalidada por evento de escrita relevante

Divergência GPS × odômetro (RN-060): job diário compara a distância derivada do GPS com a variação do odômetro no mesmo período; divergência acima de 5% gera flag de auditoria visível ao gestor.

DAT-03 Cloudflare R2 para objetos

Justificativa. O RNF-022 exige URLs assinadas com expiração máxima de 15 minutos, o que implica **reassinatura e download frequentes**. Com egress cobrado (S3: US\$0,09/GB), o custo se torna imprevisível. O R2 tem **egress gratuito** e API compatível com S3, permitindo uso do SDK AWS padrão em Java.

Estrutura de buckets

Trecho de referência 17 — Estrutura de buckets

```
rookhub-media/                (privado, presigned URLs)
  {tenant_id}/checklists/{ano}/{mes}/{checklist_id}/{item_id}/{uuid}.webp
  {tenant_id}/receipts/{ano}/{mes}/{fueling_id}/{uuid}.webp
  {tenant_id}/orders/{os_id}/{uuid}.{ext}

rookhub-safety/               (privado, retenção 90 dias - RN-093)
  {tenant_id}/events/{ano}/{mes}/{dia}/{event_id}/frame.webp

rookhub-datasets/             (privado, ativo estratégico - RN-082)
  {tenant_id}/labeled/{ano}/{mes}/frames/{event_id}.webp
  {tenant_id}/labeled/{ano}/{mes}/labels.parquet

rookhub-imports/              (privado, TTL 30 dias)
  {tenant_id}/{import_id}/{arquivo}
```

Upload de mídia: o cliente solicita uma presigned URL ao backend (que valida tenant, papel e entitlement) e envia diretamente ao R2, sem trafegar pela aplicação. Reduz custo de computação e latência.

Política de retenção (RN-145) aplicada por lifecycle rules e por job do worker, com registro auditado de expurgo.

DAT-04 Copiloto do operador sem pipeline de vídeo

Contexto. A RN-080 exige priorizar as N câmeras com maior probabilidade de evento, mas a RN-092 é explícita: **o RookHub não armazena mídia**, apenas metadados e URL assinada apontando para o fornecedor.

Decisão. Priorização por **sinais não-visuais**. O RookHub **não constrói pipeline de vídeo próprio**.

Modelo de score de priorização

O ranking de câmeras é calculado a partir de sinais já disponíveis:

Quadro 21 — Modelo de score de priorização

SINAL	ORIGEM	PESO INDICATIVO
Frenagem brusca / aceleração anômala	Telemetria (Powerfleet, Eagle-track)	Alto
Tempo de condução contínua	Viagens + telemetria	Alto
Horário de risco (madrugada)	Relógio	Médio
Score de segurança do motorista (RF-031)	Histórico interno	Médio
Histórico de eventos nas últimas 24h	Eventos internos	Alto
Velocidade acima do padrão do trecho	Telemetria	Médio
Excesso de jornada	Viagens	Alto

Fonte: elaboração própria, RookHub (2026).

O operador vê as **N câmeras mais prováveis** em destaque; as demais permanecem acessíveis sob demanda. A decisão permanece humana (RN-080).

Vídeo é buscado sob demanda, apenas quando o operador clica, via URL assinada do Hik-Connect.

Alternativa descartada — **RT-03**. Um pipeline próprio de vídeo (RTSP → ingest → inferência) foi **formalmente rejeitado**. Ele transformaria a RookHub em empresa de infraestrutura de mídia, contradizendo a mesma justificativa que a RN-086 usa para sus-

pendar o desenvolvimento de hardware. É o maior risco de escopo do projeto e deve ser recusado sempre que reproposto.

Caminho preferencial pós-MVP. Assim que [PRM-001](#) for validado, migrar para o **consumo de eventos de IA do fornecedor** (Powerfleet Unity já comercializa vídeo com IA e alerta ao motorista). Isso elimina a necessidade de análise própria e é o caminho que o próprio PRD coloca em avaliação em [SEG-13](#).

DAT-05 Dataset de treinamento (RN-082)

O PRD declara o par (frame analisado, decisão do operador) como **ativo estratégico** do módulo de segurança.

Decisão. Abordagem dupla:

1. **PostgreSQL como índice** — tabela [training_labels](#) com [event_id](#), [tenant_id](#), [frame_url](#), decisão do operador (confirmado / descartado), timestamp, operador responsável, tipo de evento, e vínculo com contestações (RN-097: evento descartado é marcado como falso positivo)
2. **Parquet no R2** — job mensal do worker exporta o dataset em formato colunar particionado por ano/mês, pronto para consumo em pipeline de treinamento **[Fase 2]**

Consentimento e LGPD. O uso do dataset para treinamento deve estar coberto pelo termo de ciência (RN-144) e pelo RIPD, com validação jurídica antes do go-live.

DAT-07 Log de auditoria imutável (RNF-020)

Eventos auditados: acesso cross-tenant, liberação de veículo, reset de PIN, edição e exclusão de lançamento, decisão de contestação, reabertura de período.

Consulta SQL 18 — Log de auditoria imutável (RNF-020)

```
CREATE TABLE audit_log (  
  id          BIGSERIAL PRIMARY KEY,  
  tenant_id   UUID          NOT NULL,  
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(),  
  actor_id    UUID          NOT NULL,  
  action      TEXT          NOT NULL,  
  entity_type TEXT          NOT NULL,  
  entity_id   UUID,  
  old_value   JSONB,  
  new_value   JSONB,  
  reason      TEXT,  
  ip_address  INET,  
  prev_hash   TEXT,  
  row_hash    TEXT          NOT NULL  
);
```



```
CREATE RULE audit_log_no_update AS ON UPDATE TO audit_log DO INSTEAD NOTHING;
CREATE RULE audit_log_no_delete AS ON DELETE TO audit_log DO INSTEAD NOTHING;
```

Hash encadeado. Cada registro carrega `row_hash = SHA256(prev_hash || conteúdo canônico)`. Qualquer adulteração quebra a cadeia e é detectável por verificação periódica. Fornece prova de não-adulteração sem custo de infraestrutura adicional.

Retenção: 5 anos (RN-145).

5.1 Modelo de dados — convenções obrigatórias

Quadro 22 — Modelo de dados — convenções obrigatórias

CONVENÇÃO	REGRA
Tenant	Toda tabela de domínio tem <code>tenant_id UUID NOT NULL</code> + policy RLS. Verificado no CI (RN-002).
Chaves	UUID v7 (ordenável por tempo, melhor localidade de índice que UUID v4)
Soft delete (RN-134)	<code>deleted_at TIMESTAMPTZ NULL</code> — exclusão física nunca ocorre em dado financeiro
Versionamento (RN-133)	Tabelas <code>*_history</code> com autor, timestamp, valor anterior e valor novo
Moeda (RN-132)	Campo <code>currency CHAR(3) NOT NULL DEFAULT 'BRL'</code> presente desde o dia 1
Valores monetários	<code>NUMERIC(18,4)</code> — nunca ponto flutuante
Tempo (RNF-017)	Armazenamento em UTC (<code>TIMESTAMPTZ</code>); apresentação em <code>America/Sao_Paulo</code>
Entitlement por veículo (RN-005)	<code>tenant_modules.vehicle_id UUID NULL</code> desde o MVP, evitando migration destrutiva na Fase 2
Template versionado (RN-033)	<code>checklist_template_version_id</code> gravado em cada checklist preenchido
Dois timestamps (RN-054)	<code>filled_at</code> (dispositivo) e <code>received_at</code> (servidor) em registros originados offline

Fonte: elaboração própria, RookHub (2026).

6 Bloco 4 — Inteligência Artificial

IA-01 Google Gemini via API direta (camada paga)

Decisão. Gemini API com chave paga (Google AI Studio), atendendo integralmente à **RN-110** — camada gratuita é proibida em **todos** os ambientes que tratem dado real de cliente, inclusive desenvolvimento.

Migração planejada para OpenAI. O tomador de decisão registrou intenção de migrar para um modelo OpenAI no futuro. Isso eleva a **RN-109** (abstração `LLM_PROVIDER`) de boa prática a **requisito de curto prazo**: nenhuma regra de negócio pode depender de particularidade de um modelo específico.

Vertex AI foi considerado e descartado no MVP por exigir uma segunda nuvem (GCP) além da AWS. Fica registrado como caminho de evolução se um cliente enterprise exigir garantia contratual de residência de dados no Brasil (`southamerica-east1`) — argumento comercial relevante em vendas para transportadoras de grande porte.

Modelo inicial: Gemini 2.5 Flash (equilíbrio custo/latência adequado ao RNF-002 de < 4s no p95). Escalonamento para modelo superior apenas na intenção 10 (generativa), se a qualidade exigir.

IA-02 Orquestração com LangChain4j

Justificativa. Maturidade superior ao Spring AI em function calling, que é o coração da RN-107. Integração com Spring Boot disponível. Abstração de provedor pronta, atendendo à RN-109 — a troca de Gemini por OpenAI passa a ser configuração, não refatoração.

Trecho Java 19 — Orquestração com LangChain4j

```
public interface LlmProvider {
    LlmResponse ask(LlmRequest request);
    Stream<String> askStreaming(LlmRequest request);
}
```

Implementações: `GeminiProvider` (MVP), `OpenAiProvider` (planejado). Seleção por propriedade `rookhub.llm.provider`.

IA-04 Pipeline do assistente e governança

Arquitetura (RN-107: text-to-SQL proibido)

O assistente **não gera SQL**. Ele classifica a intenção e invoca funções Java auditadas, previamente registradas, correspondentes ao catálogo de 10 intenções da RF-034.

Trecho Java 20 — Arquitetura (RN-107: text-to-SQL proibido)

```
@Tool("Retorna o ranking de custo por quilômetro dos veículos no período")
@RequiresEntitlement(Module.COSTS)
@RequiresFinancialVisibility
public CostRankingResult costPerKmRanking(LocalDate from, LocalDate to) { ... }
```

Justificativa (RN-108). Em demonstração comercial, "ainda não sei responder isso" é infinitamente preferível a um número errado. A precisão numérica é o ativo do produto.

Gate de autorização (RN-118, RN-119) — requisito bloqueante

Trecho de referência 21 — Gate de autorização (RN-118, RN-119) — requisito bloqueante

```
[1] Classificação de intenção + function calling (mesma chamada)
    ↓
[2] GATE – executado no backend, ANTES de invocar a função:
    • O plano do tenant inclui o módulo?      (tenant_modules)
    • O papel do usuário permite a ação?      (matriz RBAC)
    • operator_sees_financials está ativa?    (RF-007)
    x ⇒ a função NÃO é chamada; recusa educada, sem revelar a existência do dado
    ↓
[3] Execução da função (j00Q, dentro de withTenant)
```

A verificação jamais ocorre por instrução no prompt. Filtro por prompt não é controle de acesso (RN-119). Se o dado não pode ser visto, ele **não entra no contexto do modelo** — a garantia é estrutural, não persuasiva.

Tokenização do payload (RN-122)

Decisão: tokenização **determinística por hash com salt por tenant**, com mapa reverso mantido apenas em memória, no escopo da requisição.

Trecho de referência 22 — Tokenização do payload (RN-122)

```
"João Silva" → DRV_7f3a2b (SHA256(salt_tenant || driver_id), truncado)
"ABC1D23"    → VEH_2b91c4
CPF, telefone → removidos integralmente do payload
```

Quadro 23 — Tokenização do payload (RN-122)

PROPRIÉ-DADE	BENEFÍCIO
Determinís-tico	O mesmo motorista recebe o mesmo token entre turnos, preservando a coerência da memória de 5 turnos (RN-117) sem persistir mapa algum
Salt por te-nant	Impossibilita correlação entre tenants
Memória apenas	Nada é gravado; o mapa reverso morre com a requisição

Fonte: elaboração própria, RookHub (2026).

A re-hidratação ocorre no backend, antes de a resposta chegar ao usuário.

Demais regras de governança

Quadro 24 — Demais regras de governança

REQUISITO	IMPLEMENTAÇÃO
RN-120 — escopo te-mático fechado	System prompt restritivo + validação de intenção; assuntos fora do catálogo recebem recusa educada
RN-121 — fonte e pe-ríodo	Toda resposta inclui metadados da consulta: "com base em 342 abastecimentos, jan-mar/2026"
RN-116 — ação con-textual	A resposta carrega <code>suggestedActions</code> estruturadas, renderizadas como botões
RN-117 — memória	5 turnos em Redis com TTL de sessão; histórico persistido 30 dias (RN-145)
RN-123 — consumo	Tokens por tenant registrados em métrica dedicada, para monitoramento de margem
RN-124 — lacunas	Pergunta não atendida é registrada em <code>assistant_gaps</code> , alimentando o ro-admap por uso real
RN-111 — intenção generativa	A intenção 10 recebe exclusivamente dados retornados pelas funções de-terminísticas e é apresentada como sugestão, nunca como afirmação

Fonte: elaboração própria, RookHub (2026).

IA-03 Voz: Google STT + ElevenLabs TTS

Pipeline em três etapas conforme RN-112: STT → LLM com function calling → TTS. Áu-dio nativo em tempo real fica para reavaliação pós-MVP (custo ~10× superior).

Quadro 25 — Voz: Google STT + ElevenLabs TTS

ETA-PA	ESCOLHA	JUSTIFICATIVA
STT	Google Speech-to-Text	Excelente qualidade em pt-BR, latência baixa, coerente com o Gemini. Modelo <code>latest_short</code> para comandos curtos.
TTS	ElevenLabs	Qualidade de voz superior. A funcionalidade é vitrine comercial (exclusiva de <code>OWNER</code> / <code>MANAGER</code> , RN-113), e voz robótica comprometeria a demonstração.

Fonte: elaboração própria, RookHub (2026).

Streaming obrigatório. O RNF-003 exige início da fala em menos de 6s no p95. A síntese deve começar assim que os primeiros tokens da resposta estiverem disponíveis, não após a resposta completa.

A resposta sempre aparece na tela (RN-114), mesmo quando falada.

RT-04 **Custo do TTS**

ElevenLabs custa aproximadamente **10× mais** que Google Cloud TTS ou AWS Polly (a partir de US\$99/mês no plano Creator, contra ~US\$16/1M caracteres).

Mitigações: - Cache de respostas sintetizadas frequentes (saudações, recusas, mensagens de erro) - Limite de caracteres por resposta falada - Métrica dedicada de custo de voz por tenant - **Gatilho de reavaliação:** se o custo de voz ultrapassar 15% da fatura mensal de infraestrutura, migrar para Google TTS. A abstração de provedor deve estar pronta desde o MVP.

IA-05 **Vector DB: pgvector, ativação condicional**

Decisão. Nenhum vector database dedicado no MVP. Se busca semântica se mostrar necessária, usar **pgvector** no PostgreSQL já existente.

Justificativa. O PRD proíbe text-to-SQL (RN-107) e fecha o escopo temático (RN-120). Não há base documental a recuperar — o assistente responde sobre dados estruturados da frota. RAG documental é, portanto, desnecessário.

Usos potenciais futuros: busca semântica no catálogo de intenções (para melhorar a sugestão de perguntas próximas da RN-124) e base de conhecimento de manutenção.

Pinecone, Qdrant e similares (US\$70+/mês) foram descartados por não haver caso de uso que justifique um serviço adicional.

7 Bloco 4 — Infraestrutura e DevOps

INF-01 AWS ECS Fargate + ALB

Quadro 26 — AWS ECS Fargate + ALB

SERVIÇO	TASKS (MVP)	RECURSOS	ESCALONAMENTO
<code>api</code>	2 (mínimo, alta disponibilidade)	1 vCPU / 2 GB	CPU > 70% ou conexões WS
<code>worker</code>	1	1 vCPU / 2 GB	Profundidade da fila

Fonte: elaboração própria, RookHub (2026).

Justificativa. A escolha por WebSocket (`FE-11`) eliminou serverless. Fargate oferece o melhor equilíbrio entre custo e carga operacional para um time de 3 desenvolvedores: sem gerenciamento de instâncias, patches ou failover manual.

ALB: terminação TLS 1.2+ (RNF-018), suporte nativo a WebSocket, sticky sessions, health checks, AWS WAF na frente.

EKS/Kubernetes descartado — control plane a US\$73/mês mais complexidade que exigiria dedicação quase integral de um desenvolvedor. Reavaliar apenas acima de 50 tenants.

Região: `sa-east-1` (São Paulo) — menor latência para os clientes e argumento de residência de dados sob LGPD.

INF-02 Hospedagem dos frontends

Quadro 27 — Hospedagem dos frontends

APP	HOSPEDAGEM	CUSTO
<code>apps/web</code>	Cloudflare Pages	US\$0
<code>apps/driver</code>	Cloudflare Pages	US\$0
<code>apps/site</code>	Vercel	US\$20/seat

Fonte: elaboração própria, RookHub (2026).

Justificativa. A conta Cloudflare já existe em função do R2 (`DAT-03`), e o egress gratuito importa especialmente para o PWA sincronizando fotos em campo. O site institu-

cional fica na Vercel, onde o SSR do Next.js funciona sem atrito e o time de marketing consegue operar com autonomia.

Atenção ao service worker: os headers de cache do PWA exigem configuração específica — `sw.js` com `Cache-Control: no-cache` e assets versionados com cache longo. Erro aqui causa aplicação travada em versão antiga, defeito difícil de diagnosticar em campo.

INF-03 CI/CD, IaC, ambientes e migrations

Quadro 28 — CI/CD, IaC, ambientes e migrations

ITEM	ESCOLHA
CI/CD	GitHub Actions — OIDC para AWS (sem chave estática), matriz de jobs, cache de dependências
IaC	Terraform / OpenTofu — multi-provider (AWS + Cloudflare + Vercel + Timescale), state remoto em S3 com lock em DynamoDB
Ambientes	<code>staging</code> + <code>production</code> — sem ambiente <code>dev</code> compartilhado; desenvolvimento local com Docker Compose
Migrations	Flyway — SQL puro, versionado, previsível

Fonte: elaboração própria, RookHub (2026).

Justificativa do Terraform sobre CDK. A infraestrutura não é exclusivamente AWS: envolve Cloudflare (R2, Pages), Vercel e Timescale Cloud. O CDK cobriria apenas a parcela AWS, deixando o restante fora do controle de versão — inaceitável quando o RNF-024 exige RTO de 4h, o que pressupõe capacidade de recriar a infraestrutura rapidamente.

RT-05 Ausência de ambiente de desenvolvimento compartilhado

A opção por apenas `staging` + `production` reduz custo, mas concentra risco: `staging` passa a ser simultaneamente ambiente de integração, testes de QA e demonstração comercial.

Mitigações: - Docker Compose local completo (Postgres+Timescale, Redis, MinIO como substituto do R2) - Testcontainers em todos os testes de integração - Seeds determinísticos permitindo reset de `staging` a qualquer momento - Janela protegida antes de demonstrações comerciais

Pipeline

Diagrama 23 — Pipeline

```

Pull Request
├── lint + typecheck (frontends)
├── build Java + testes unitários
├── Testcontainers: integração (Postgres + Timescale + Redis)
├── ArchUnit: fronteiras do Modulith + proibição de DataSource fora do TenantContext
├── GATE RN-002: toda tabela de domínio tem tenant_id NOT NULL + policy RLS [BLOQUEANTE]
├── GATE cross-tenant: teste de concorrência prova ausência de vazamento [BLOQUEANTE]
├── GATE RNF-028: axe-core (contraste WCAG AA) [BLOQUEANTE]
├── GATE RNF-006: teste cronometrado de esforço do operador [BLOQUEANTE]
├── Trivy: vulnerabilidades em dependências e imagem
└── Lighthouse CI: performance do PWA (bundle budget)

Merge em main
├── build da imagem Docker → ECR
├── Flyway migrate em staging
├── deploy em staging (rolling)
├── smoke tests
└── deploy em production (aprovação manual)
  
```

Gates bloqueantes — justificativa

Quadro 29 — Gates bloqueantes — justificativa

GATE	ORIGEM	POR QUE É BLOQUEANTE
RLS e tenant_id	RN-002	O PRD determina explicitamente que o build falhe e que a regra não possa ser desativada por decisão de squad
Vazamento cross-tenant	RN-001	Defeito que só se manifesta sob concorrência; sem teste automatizado, chega a produção
Contraste WCAG AA	RNF-028	Risco inerente ao glassmorphism; verificação manual não escala
Esforço do operador	RNF-006	Tradução mensurável do princípio inegociável; o PRD determina que regressão bloqueie o deploy

Fonte: elaboração própria, RookHub (2026).

INF-04 Observabilidade e serviços de apoio

Quadro 30 — Observabilidade e serviços de apoio

NECESSIDADE	ESCOLHA	OBSERVAÇÃO
APM, métricas, logs	Datadog	Item mais caro da infraestrutura — ver RT-06
Rastreio de erros	Sentry	Backend Java + 3 frontends, com session replay

NECESSIDADE	ESCOLHA	OBSERVAÇÃO
E-mail transacional (RN-048)	AWS SES	~US\$0,10 por mil e-mails
Push (RN-048)	Firebase Cloud Messaging	Web Push no PWA; caminho pronto para o app nativo [Fase 2]
Ligação automática (RN-085)	Zenvia	Fornecedor nacional, faturamento e fiscal simplificados no Brasil
Secrets	AWS Secrets Manager	Rotação automática, integração nativa com ECS

Fonte: elaboração própria, RookHub (2026).

RT-06 Custo do Datadog

O Datadog é excelente, mas seu custo (~US\$150–250/mês com APM, logs e infraestrutura) supera o do banco de dados no estágio atual, representando o maior item isolado da fatura.

Mitigações: - Iniciar com plano **Pro**, apenas **APM + Logs**, sem RUM e sem Synthetics - Amostragem de traces em 10% para rotas de alto volume - Retenção de logs em 7 dias (auditoria de longo prazo vive no `audit_log`, não no Datadog) - Filtragem de logs de baixo valor antes do envio - **Gatilho de reavaliação:** se o custo ultrapassar 20% da fatura mensal, migrar para Grafana Cloud (o instrumento é OpenTelemetry, o que torna a migração de baixo custo)

Instrumentação: **OpenTelemetry** em todo o backend, jamais o agente proprietário diretamente. Isso mantém a portabilidade entre plataformas de observabilidade.

Alertas mínimos

Quadro 31 — Alertas mínimos

ALERTA	LIMIAR	ORIGEM
Disponibilidade	< 99,5% mensal	RNF-008
Painel do Dono p95	> 2s	RNF-001
Assistente de IA p95	> 4s	RNF-002
Salvamento do operador p95	> 500ms	RNF-004
Evento crítico → toast	> 10s	RNF-007
Integração sem sincronizar	> 30 min	RN-140

ALERTA	LIMIAR	ORIGEM
Profundidade de fila	> 1000 mensagens	—
Dead letter queue	qualquer mensagem	—
Custo diário de LLM/TTS	acima do orçado	RN-123
Falha de verificação da cadeia de hash	qualquer	RNF-020

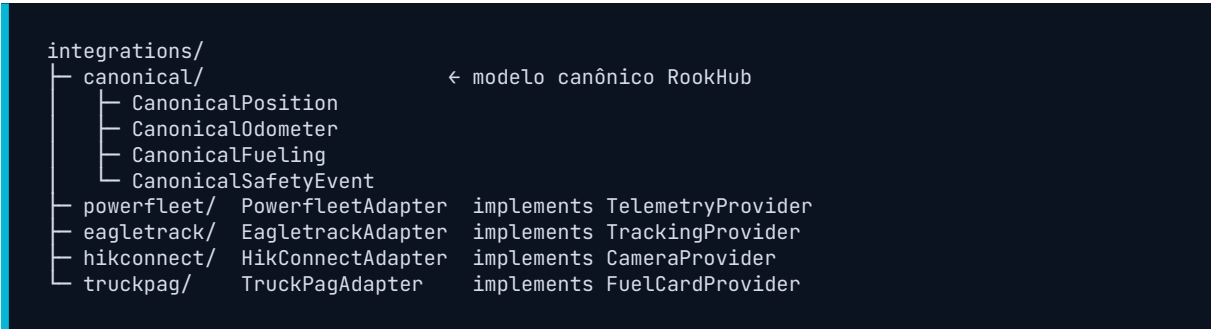
Fonte: elaboração própria, RookHub (2026).

8 Integrações

INT-01 Camada de anticorrupção obrigatória (RN-138)

Requisito **arquiteturalmente bloqueante** (RN-139): sem ele, o segundo cliente com outro rastreador implicaria reescrita do núcleo.

Diagrama 24 — Camada de anticorrupção obrigatória (RN-138)



Nenhuma regra de negócio conhece o formato de um fornecedor específico. Teste ArchUnit garante que classes fora do pacote `integrations.{fornecedor}` não referenciem tipos daquele fornecedor.

INT-02 Padrão híbrido (RN-137)

Quadro 32 — Padrão híbrido (RN-137)

MECANISMO	USO
Webhook	Quando o fornecedor suportar — caminho preferencial (menor latência, atende RNF-007)
Polling agenda-do	Complemento e fallback, com intervalo configurável por fornecedor
Retry exponen-cial	1s → 32s, 6 tentativas, seguido de DLQ com alerta

Fonte: elaboração própria, RookHub (2026).

INT-03 Observabilidade de integração

Quadro 33 — Observabilidade de integração

REQUISITO	IMPLEMENTAÇÃO
RN-140	<code>last_successful_sync_at</code> por fornecedor e por tenant, em tabela dedicada
RN-141	Banner de dado desatualizado no painel, exibindo o horário da última sincronização bem-sucedida
RN-142	Idempotência por <code>(tenant_id, provider, external_event_id)</code> com constraint única

Fonte: elaboração própria, RookHub (2026).

DECISÃO

O RN-141 é requisito de confiança, não de conveniência: o dono precisa saber que está olhando um número velho antes de decidir com base nele.

9 Segurança

Quadro 34 — Segurança

REQUISITO	IMPLEMENTAÇÃO
RNF-018 — TLS 1.2+ e criptografia em repouso	TLS terminado no ALB (política mínima 1.2); criptografia em repouso no Timescale Cloud, R2 e ElastiCache
RNF-019 — Argon2id	<code>Argon2PasswordEncoder</code> para senhas e PINs
RNF-020 — Auditoria imutável	<code>DAT-07</code> (append-only + hash encadeado)
RNF-021 — Rate limiting	Redisson, por tenant e por usuário, em todos os endpoints públicos
RNF-022 — URLs assinadas	Expiração máxima de 15 minutos, geradas sob demanda após validação de tenant, papel e entitlement
RN-001/002/003 — Isolamento	<code>BE-14</code> (RLS + quatro camadas de garantia)

Fonte: elaboração própria, RookHub (2026).

Práticas adicionais:

- Segredos exclusivamente no Secrets Manager; **nunca** em variáveis de ambiente de imagem ou em repositório
- Imagens Docker escaneadas por Trivy no CI; base distroless
- Dependências monitoradas por Dependabot
- CSP restritiva nos frontends
- Princípio do menor privilégio nas IAM roles (uma por serviço)
- Aplicação conecta ao banco com usuário **sem** `BYPASSRLS`

9.1 LGPD (Seção 19 do PRD)

Quadro 35 — LGPD (Seção 19 do PRD)

REQUISITO	IMPLEMENTAÇÃO
RN-143 — legítimo interesse	Base legal documentada; RIPD e política de privacidade pendentes de validação jurídica antes do go-live
RN-144 — termo de ciência	Exibido no primeiro acesso ao PWA; aceite registrado com data, versão do termo e IP

REQUISITO	IMPLEMENTAÇÃO
RN-145 — retenção	Jobs de expurgo no worker, por categoria, com registro auditado
RN-146 — anonimização	Job mensal anonimiza nome e CPF de motorista desligado há 12+ meses, preservando integralmente as métricas agregadas
RN-147 — direitos do titular	Processo manual via suporte no MVP; portal automatizado [Fase 2]
RN-122 — dados na IA	Tokenização antes do envio ao provedor (IA-04)

Fonte: elaboração própria, RookHub (2026).

DESTAQUE

Aviso. Este documento não constitui aconselhamento jurídico. O RIPD, a política de privacidade e o termo de ciência devem ser validados por advogado especializado em proteção de dados antes do go-live, conforme determina o próprio PRD.

10 Custos Operacionais Estimados

10.1 MVP — 2 tenants, ~80 veículos

Quadro 36 — MVP — 2 tenants, ~80 veículos

ITEM	FORNECEDOR	FAIXA MENSAL (USD)
Banco de dados	Timescale Cloud	50 – 150
Computação (api + worker)	AWS ECS Fargate	60 – 100
Load balancer	AWS ALB	20 – 25
Cache / filas	AWS ElastiCache (t4g.micro)	15 – 25
Object storage (~500 GB)	Cloudflare R2	8
Frontends	Cloudflare Pages + Vercel	0 – 20
LLM	Google Gemini	20 – 80
STT	Google Speech-to-Text	5 – 20
TTS	ElevenLabs	99 – 150
Observabilidade	Datadog	150 – 250
Rastreo de erros	Sentry	26
E-mail transacional	AWS SES	1 – 5
Push	Firebase FCM	0
Ligações automatizadas	Zenvia	10 – 40
Mapas	Mapbox	0 – 50
Secrets, ECR, egress, diversos	AWS	10 – 20
Total estimado		≈ US\$ 475 – 970

Fonte: elaboração própria, RookHub (2026).

10.2 Análise

Os dois maiores itens da fatura — **Datadog e ElevenLabs, somando US\$250–400** — não são infraestrutura de produto, mas escolhas de qualidade. Ambos possuem gatilho de reavaliação documentado ([RT-04](#) , [RT-06](#)) e alternativas de custo aproximadamente 10× menor, com migração de baixo custo.

Reduzindo esses dois itens às alternativas econômicas (Grafana Cloud + Google TTS), o custo total cairia para aproximadamente **US\$250–450/mês**.

Projeção para 10 tenders / ~800 veículos: estimativa de US\$1.200–2.000/mês, com crescimento predominantemente em banco de dados, computação e LLM. Custo por tenant **cai** com a escala — característica saudável do modelo.

11 Riscos Técnicos e Dívidas Assumidas

Quadro 37 — Riscos Técnicos e Dívidas Assumidas

ID	RISCO / DÍVIDA	SEVE- RIDADE	MITIGAÇÃO
RT-01	Limitações de PWA em iOS (push, cota de IndexedDB, expurgo do Safari)	Alta	Levantar distribuição Android/iOS antes da sprint 1; fallback por SMS; solicitar armazenamento persistente
RT-02	Custo do Mapbox escalando de forma não-linear	Média	Instância única de mapa; atualização por setData ; alerta de orçamento; migração para MapLibre acima de US\$100/mês
RT-03	Pressão futura por pipeline de vídeo próprio	Alta	Formalmente rejeitado. Transformaria a RookHub em empresa de infraestrutura de mídia, contradizendo a justificativa da RN-086
RT-04	Custo do TTS ElevenLabs (~10x alternativas)	Média	Cache de respostas frequentes; abstração de provedor; gatilho de migração em 15% da fatura
RT-05	Ausência de ambiente de desenvolvimento compartilhado	Média	Docker Compose local completo; Testcontainers; seeds determinísticos; janela protegida antes de demonstrações
RT-06	Custo do Datadog superior ao do banco de dados	Média	Plano Pro sem RUM; amostragem de traces; instrumentação via OpenTelemetry garante portabilidade
RT-07	Performance do glass-morphism em hardware modesto	Média	Fallback automático de blur (FE-07); proibição de blur por item em listas longas
RT-08	Fatura separada do Timescale Cloud (fora do consolidado AWS)	Baixa	Aceito conscientemente; alternativa (RDS puro) traria dívida técnica maior
RT-09	Dependência de fornecedores de telemetria para a proposta de valor central	Alta	ACL obrigatória (RN-138); banner de dado desatualizado (RN-141); last_successful_sync_at monitorado
RT-10	LangChain4j e Spring AI ainda em maturação	Baixa	Interface LlmProvider própria isola a aplicação da biblioteca
DT-01	Migração planejada Gemini → OpenAI	—	Abstração LLM_PROVIDER desde o dia 1 (RN-109)

ID	RISCO / DÍVIDA	SEVE- RIDADE	MITIGAÇÃO
DT-02	Kafka adiado para a Fase 2	—	Camada de filas abstraída por interface desde o MVP
DT-03	Python restrito à Fase 2 (visão computacional)	—	Dataset já persistido em Parquet desde o MVP (DAT-05)

Fonte: elaboração própria, RookHub (2026).

12 Rastreabilidade — Decisão × Requisito

Quadro 38 — Rastreabilidade — Decisão × Requisito

DECISÃO	REQUISITOS ATENDIDOS
FE-01 , FE-02 , FE-12	RNF-026, RNF-027
FE-03 , FE-04	RNF-015, RNF-001
FE-05	RNF-014, RNF-029, RN-041
FE-06	RNF-026
FE-07	RNF-028, RNF-006
FE-08	RN-052, RN-053, RN-054, RNF-009 a RNF-013
FE-09	RF-032, RN-116
FE-11	RN-087, RN-091, RNF-007
BE-04	RF-042, RN-118
BE-09 , BE-08	RNF-004, RNF-008
BE-10 , DAT-06	RNF-001, RF-018
BE-11	RF-003 a RF-007, RNF-012, RNF-019, RNF-021
BE-12	RN-130, RN-137, RN-142
BE-14	RF-001, RF-002, RN-001 a RN-005
DAT-01a , DAT-02	RF-019, RNF-001, RNF-023, RNF-024
DAT-03	RN-040, RNF-022, RN-145
DAT-04	RN-080, RN-092
DAT-05	RN-082, RN-097
DAT-07	RNF-020, RN-133, RN-134
IA-01 , IA-02	RN-107, RN-109, RN-110
IA-03	RN-112, RN-113, RN-114, RNF-003

DECISÃO	REQUISITOS ATENDIDOS
IA-04	RN-118 a RN-124, RNF-002
INF-01	RNF-008, RNF-018
INF-03	RN-002, RNF-006, RNF-024, RNF-028
INF-04	RN-048, RN-085, RN-140
INT-01 a INT-03	RF-041, RF-042, RN-137 a RN-142

Fonte: elaboração própria, RookHub (2026).

13 Definição de Pronto — Complemento Técnico

Além dos critérios do Anexo G do PRD, nenhuma funcionalidade é considerada pronta sem:

- [] Tabelas novas com `tenant_id NOT NULL` e policy RLS (gate de CI aprovado)
- [] Acesso a dados exclusivamente através de `TenantContext.withTenant()`
- [] Teste de integração com Testcontainers cobrindo o caminho feliz e o de erro
- [] Teste ArchUnit aprovado (fronteiras de módulo preservadas)
- [] Endpoint documentado no OpenAPI e cliente TypeScript regenerado
- [] Verificação de entitlement e de permissão RBAC aplicada no backend
- [] Operações auditáveis gravando em `audit_log` com hash encadeado
- [] `axe-core` sem violações nas telas afetadas
- [] Métrica de latência instrumentada quando houver RNF associado
- [] Funcionalidade offline testada em modo avião, se aplicável ao PWA
- [] Migration Flyway versionada e reversível
- [] Nenhum segredo em código ou variável de ambiente de imagem

14 Próximos Passos Recomendados

Antes da sprint 1 (bloqueantes)

1. **Levantar a distribuição Android/iOS da frota-âncora** — `RT-01` pode alterar a estratégia de aplicativo do motorista
2. **Validar `PRM-001`** (meio físico do alerta em cabine) — o PRD determina que nenhuma sprint de hardware inicie antes disso
3. **Obter documentação de API dos 4 fornecedores** do MVP — a modelagem canônica depende do formato real
4. **Contratar validação jurídica** do RIPD e da política de privacidade

Sprint 0 — fundação técnica (2 semanas sugeridas)

1. Monorepo, Terraform e pipelines de CI/CD
2. **Esqueleto multi-tenant com RLS e os quatro gates de garantia funcionando** — antes de qualquer regra de negócio
3. `packages/tokens` e `packages/ui` com os primitivos de vidro e o fallback de blur
4. Autenticação completa (Spring Security, os três fluxos)
5. Estrutura de módulos do Spring Modulith com testes de fronteira

DESTAQUE

A recomendação mais importante deste documento: **o isolamento multi-tenant deve estar completo, testado e blindado por CI antes da primeira regra de negócio ser escrita**. Retrofit de RLS em base já povoada é a operação mais cara e arriscada dessa arquitetura.

Documento gerado a partir de sessão de arquitetura conduzida com aprovação explícita de cada decisão técnica. Revisão recomendada ao final da Sprint 0 e a cada marco de escala (10 tenants, 50 tenants).

RookHub — Arquitetura e Decisões Técnicas · Versão 1.0 · Julho de 2026.

Documentos de origem: PRD RookHub, Visão e Escopo de Negócio e Design System RookHub.